DECIDING WITHOUT GENERATING: EVALUATING TYPED DECISION MODELS IN LLM SYSTEMS

Anonymous Authors[1]

ABSTRACT

In compound AI architectures, generative large language models (LLMs) are routinely invoked not to produce freeform text, but to make discrete systems decisions: selecting API tools in agent loops, validating admissions in semantic prompt caches, and routing queries across model hierarchies. Recently, specialized non-autoregressive "decision models" (e.g., TypeSafe Jev and open reproductions such as Kev) have emerged, promising orders-of-magnitude reductions in latency and cost by directly outputting calibrated categorical distributions rather than generating tokens. This paper presents the first comprehensive, independent empirical evaluation of the deciding-without-generating paradigm across three foundational LLM-system decision points on a shared evaluation harness: agent control (τ-bench airline), semantic cache guarding (GSM-Plus, LM-Arena, and SearchQueries), and model routing (RouterBench). We measure decision accuracy, end-to-end task completion, wall-clock latency, dollar cost, and physical hardware energy on NVIDIA A100 GPUs.

Our findings establish a unifying systems principle: *typed decision models pay off if and only if their decision replaces downstream generation and its input provides semantic signal that cheap heuristics cannot resolve.* In agent control, where generation must proceed anyway to synthesize tool parameters, a unified frontier LLM performing tool choice and execution in a single call (Arm C) achieves the lowest cost ($0.033 per success) and highest reward (0.792), rendering decoupled deciders economically disadvantageous. In semantic cache guarding, where a correct decision completely skips generation, a typed decider delivers decisive gains on fine-grained reasoning shifts (GSM-Plus: $+27.2$ points accuracy over embedding thresholds, cutting system cost by 28%), while frontier LLMs acting as cache guards are economically self-defeating (costing 136% of unconditional generation). Furthermore, a two-stage cascade (similarity threshold $\rightarrow$ decider) establishes a Pareto-optimal frontier on conversational workloads, querying the decider on only 3.0% of requests. In model routing, general-purpose deciders perform near chance because routing depends on target-model error surfaces rather than generic prompt difficulty; a lightweight classifier trained directly on historical failure records recovers 64.9% of the quality gap, outperforming all general deciders ($p < 0.05$). Finally, we audit prevailing semantic caching benchmarks, demonstrating that reported model ceilings were artifacts of transitive error propagation in union-find clustering, which our direct pairwise protocol resolves.

1 INTRODUCTION

Modern enterprise AI deployments have rapidly transitioned from single-prompt interactions to *compound AI systems* (Zaharia et al., 2024). In these architectures, large language models (LLMs) operate as controllers, orchestrating complex state machines, querying external toolkits (Schick et al., 2023; Mialon et al., 2023), verifying and retrieving responses from semantic prompt caches (Schroeder et al., 2026), and dynamically routing incoming requests across heterogeneous model tiers (Chen et al., 2023; Ong et al., 2024).

A pervasive assumption in these systems is that discrete architectural decisions should be made by generative autoregressive models. When an agent decides which API tool to invoke next, or when a cache evaluates whether an incoming query is semantically equivalent to a previously served prompt, standard practice routes the request to a generative LLM. This generative paradigm incurs severe systems overhead. To produce a discrete decision—such as picking one tool index out of fifteen, or deciding binary cache reuse—the

[1]Anonymous Institution, Anonymous City, Anonymous Region, Anonymous Country. Correspondence to: Anonymous Author <anon.email@domain.com>.

Preliminary work. Under review by the Machine Learning and Systems (MLSys) Conference. Do not distribute.

generative LLM must load thousands of context tokens into key-value (KV) caches, execute full self-attention passes across the prompt history, and autoregressively sample output tokens. Consequently, a single discrete decision often incurs seconds of latency, substantial monetary cost, and significant hardware energy consumption.

Recently, an alternative architectural paradigm has emerged: *deciding without generating*. This paradigm introduces specialized, non-autoregressive decision models—exemplified by commercial offerings such as TypeSafe Jev and open reproductions such as Kev (Li et al., 2024). Rather than emitting arbitrary text, these models are trained via reinforcement learning with compiler feedback (RLCD) to directly map an arbitrary context string to calibrated probability distributions over typed discrete structures: discrete choices, bounded ordinal scores, or boolean assertions. By decoupling discrete decision-making from token generation, vendor benchmarks claim over two orders of magnitude in speedup and cost reduction.

However, these claims remain largely unverified by independent systems evaluations. Crucially, the systems tradeoffs of deploying typed decision models inside real-world compound pipelines are nuanced: (1) *Decision Quality vs. Flexibility:* Can a non-autoregressive decider match frontier LLMs on complex tool-selection logic or subtle query nuances? (2) *Physical Hardware Efficiency:* Beyond API token prices, what are the true wall-clock latencies and physical GPU energy footprints (in Joules) on enterprise accelerators? (3) *End-to-End System Utility:* In an interactive multi-turn agent loop, does offline decision accuracy translate into end-to-end task completion, or does separating decision from execution introduce compounding pipeline failures? (4) *Heuristic and Cascade Interplay:* Can cheap non-neural heuristics solve the majority of decisions, and can multi-stage cascades achieve a superior Pareto frontier?

In this paper, we present the first comprehensive, independent empirical evaluation of the deciding-without-generating paradigm across three representative LLM-system decision points on a shared evaluation harness: (1) **Agent Control** in the τ-bench airline domain (Yao et al., 2024), evaluated both offline across 1,076 verified decision states and in-the-loop across 1,250 complete multi-turn task episodes; (2) **Semantic Cache Guarding** across three diverse workloads: mathematically perturbed reasoning variants (GSM-Plus), multi-turn chat paraphrases (vCache LM-Arena), and real-world search queries (vCache SearchQueries); and (3) **Model Routing** in multi-model serving (RouterBench 0-shot), routing queries between Mixtral-8x7B and GPT-4. Across all three domains, we evaluate a consistent matrix of interchangeable deciders: Jev (commercial API), Kev (open-weights reproduction at 0.8B, 4B, and 9B scales), generative LLMs (GPT-5.6, GPT-OSS-20B, and Qwen3-8B), task-specific classifiers, and cheap non-neural baselines (cosine similarity thresholds, cross-encoders, and vCache's per-prompt error bounds).

Our empirical findings establish a unifying systems principle:

> **The Decider Payoff Principle:** *A dedicated, typed decision model pays off if and only if its decision replaces subsequent generation and its input provides semantic signal that lightweight non-neural heuristics cannot resolve.*

Specifically, our evaluation yields five key systems insights:

- **Separating Decision from Inevitable Generation Does Not Pay:** In agent control, where the generator must execute anyway to synthesize tool arguments and responses, invoking a separate decider adds an extraneous network round trip and latency. A unified frontier model that handles both decision and execution in a single call (Arm C) achieves the highest task completion (0.792 reward) at the lowest cost ($0.033 per success), outperforming decoupled pipelines.
- **Deciders Excel When Generation is Skipped:** In semantic cache guarding, where a correct reuse decision entirely eliminates expensive downstream generation, typed deciders deliver dramatic gains. On GSM-Plus, where subtle numerical changes deceive embedding similarity, Jev achieves 93.3% accuracy versus 66.1% for similarity thresholds (+27.2% paired gain, $p < 0.05$), reducing total system cost by 28%. Conversely, using a frontier LLM as a cache guard is economically self-defeating, inflating system cost to 136% of unconditional generation.
- **Model Cascades Establish the Pareto Frontier:** Deploying a two-stage cascade—routing clear cases to a zero-cost embedding threshold and reserving the typed decider for an ambiguous margin—achieves optimal efficiency. On LM-Arena chat traffic, a threshold $\rightarrow$ Jev cascade queries Jev on only 3.0% of requests, achieving 97.4% accuracy at $\$5.91 \times 10^{-7}$ per decision.
- **Routing Demands Model-Specific Error Profiling:** In model routing, predicting whether a specific model will fail cannot be solved by generic semantic difficulty. Untrained deciders barely beat a random mix (recovering 60.9% of the quality gap vs 50.0% for random), whereas a lightweight classifier trained directly on empirical failure records recovers 64.9% ($p < 0.05$).
- **Methodological Benchmark Diagnostic:** In auditing standard semantic caching benchmarks (vCache), we

discover that reported model ceilings (70–72%) are artifacts of label errors (up to 15.7%) caused by transitive closure fallacies in union-find clustering. We establish a clean, non-transitive pairwise evaluation protocol that restores empirical fidelity.

2 THE DECIDING-WITHOUT-GENERATING PARADIGM

2.1 The Architectural Abstraction

In traditional LLM pipelines, decision logic is conflated with generation. An agent loop passes an execution trace to an autoregressive model, instructing it via prompt engineering or structured JSON schemas to select a tool: $P(y_{1:T} \mid x) = \prod_{t=1}^{T} P(y_t \mid x, y_{<t})$, where $y_{1:T}$ represents emitted tokens encoding the choice and its arguments. Even when grammar-guided decoding restricts sampling to valid JSON, the model must maintain full autoregressive state across input context x.

The *deciding-without-generating* paradigm treats discrete decisions as typed classification queries. A decision model implements a unified programmatic interface:

$$\mathcal{D}(s, \mathcal{C}, \mathcal{I}) \to (c^*, \hat{p}, \mathbf{p}) \tag{1}$$

where $s \in \mathcal{S}$ is an arbitrary context string (dialogue transcript, query pair, or prompt), $\mathcal{C} = \{c_1, \ldots, c_K\}$ is a discrete set of candidate options with semantic criteria, and $\mathcal{I}$ denotes task instructions. The model directly outputs the predicted option $c^* \in \mathcal{C}$, scalar confidence $\hat{p} \in [0, 1]$, and probability vector $\mathbf{p} \in \Delta^{K-1}$.

2.2 Decider Implementations Under Test

We evaluate seven interchangeable decider categories behind this shared interface:

Commercial Typed Decider (Jev): Jev (TypeSafe, evaluated via OpenRouter, pinned to `jev-1.13.0`) is a non-autoregressive decision model trained via Reinforcement Learning from Compiler Feedback (RLCD). Billed at $0.042 per million input tokens with output free, Jev represents the primary commercial realization of the typed decision paradigm.

Open-Weights Reproductions (Kev): Kev (Li et al., 2024) is an open-source reproduction of the System-1 architecture released in September 2026. Built on a Qwen3.5 backbone with LoRA and a pointer head, Kev emits discrete choices without token generation. We evaluate **Kev-0.8B**, **Kev-4B**, and **Kev-9B**, hosted locally with prefix caching disabled to prevent synthetic inflation on repeated states.

Generative LLM Deciders: We evaluate three generative LLMs invoked via constrained tool calling or structured choice prompts: **GPT-5.6** (`gpt-5.6-sol-20260709` via OpenRouter) as the frontier ceiling; **GPT-OSS-20B** as a small hosted model; and **Qwen3-8B** served locally on an NVIDIA A100-80GB GPU via vLLM (v0.29.0) with `enable_thinking=false`.

Classifiers and Heuristic Baselines: For cache guarding and routing, we construct lightweight neural baselines by fine-tuning encoder backbones (`distilroberta-base` and `cross-encoder/quora-distilroberta-base`) on task dev splits. As non-neural baselines, we evaluate cosine similarity thresholds over `gte-large-en-v1.5` embeddings, an off-the-shelf cross-encoder (`cross-encoder/quora-roberta-large`), and vCache's per-prompt error bounds δ.

2.3 Evaluation Methodology and Systems Rigor

Hardware & Energy Protocol: Local model serving and energy metering are executed on dedicated nodes equipped with an NVIDIA A100-SXM4-80GB PCIe GPU (driver 590, CUDA 13.1) in exclusive mode. We interface directly with the NVIDIA Management Library (NVML) 64-bit cumulative energy counter (`nvmlDeviceGetTotalEnergyConsumption`) to integrate hardware power at millijoule resolution. We measure *block energy*: the total cumulative energy consumed across a contiguous, metered run of N decisions divided by N. Net energy subtracts baseline idle power (61.0–71.6 W): $E_{\text{net}} = (\Delta E_{\text{total}}/N) - P_{\text{idle}} \cdot \bar{t}_{\text{decision}}$.

Statistical Bootstrapping & Calibration: All confidence intervals are computed using **cluster bootstrapping** with 1,000 resamples clustered at the natural unit of independence (the task in agent control, query class in cache guarding, prompt in routing). Calibration is quantified via Expected Calibration Error (ECE) and Brier score across 10 equally spaced confidence bins.

3 DECISION POINT 1: AGENT CONTROL

In agentic workflows, an LLM interacts with an environment across multiple turns, selecting tools, reading returns, and deciding when to synthesize final responses (Yao et al., 2024). A core hypothesis of the deciding-without-generating paradigm is that decoupling tool choice from argument generation can accelerate agent loops and reduce execution costs.

3.1 Experimental Setup: Tau2-Bench Airline

We evaluate agent control on the airline domain of τ-bench (v1.0.1, pinned to commit `fc0055d`) (Yao et al., 2024). The airline benchmark simulates complex, multi-turn customer service interactions involving

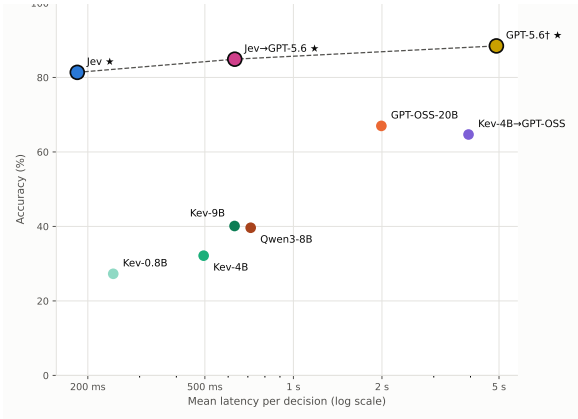


Figure 1. Decision accuracy versus mean latency Pareto frontier in offline agent control (Arm 0). Jev operates at 184 ms mean latency and 81.4% accuracy, outperforming all small LLMs and open deciders.

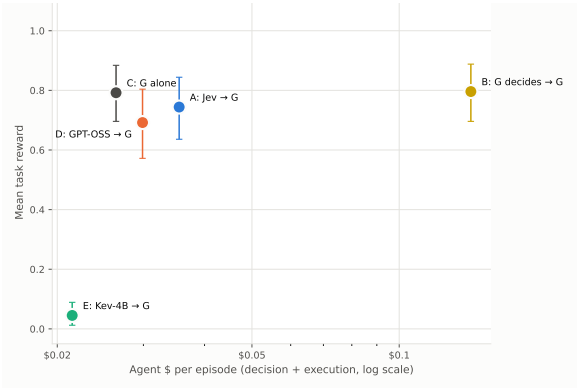


Figure 2. Task reward versus agent cost per successful task (50 tasks × 5 trials, 250 episodes per arm). Arm C (unified G alone) achieves the Pareto-optimal operating point: $0.033 per success at 0.792 reward.

50 distinct real-world tasks (e.g., flight booking, modification, seat selection, and policy cancellations). The environment provides 15 external API tools partitioned into read-only queries (e.g., `get_flight_details`, `get_reservation`) and state-modifying actions (e.g., `cancel_reservation`, `book_flight`).

We structure our evaluation into two complementary tracks:

1. **Offline Decision Replay (Arm 0):** To isolate pure decision quality, latency, cost, and energy from environmental stochasticity, we harvest 1,076 unique dialogue states from verified, reward-1.0 trajectories generated by a frontier model. Each decider evaluates every state with 5 independent repeats (5,380 decisions per decider).

2. **In-the-Loop Execution (Arms A–E):** To measure end-to-end task completion, we deploy deciders inside live, multi-turn agent loops against a simulated user across 50 tasks with 5 trials per task (250 episodes per arm).

3.2 Offline Decision Replay (Arm 0)

Table 1 presents the offline replay benchmark. We report both **strict accuracy** (exact match with reference tool) and **lenient accuracy** (which permits exploratory read-only lookups preceding state-modifying actions).

Accuracy vs. Latency Pareto Frontier: As visualized in Figure 1, Jev achieves 81.4% strict accuracy [79.1%, 83.6%] with a median latency of 176 ms and a monetary cost of $\$1.73 \times 10^{-4}$ per decision. Compared to the frontier ceiling model (GPT-5.6 at 88.5% accuracy), Jev is **22.2× faster** in median latency (176 ms vs. 3,909 ms) and **22.1× cheaper** per call ($0.00017 vs. $0.00383). Paired comparison confirms GPT-5.6's accuracy advantage is +7.1 points [+5.2, +9.1], but at an extreme latency and monetary surcharge.

Calibration and Reliability: Jev exhibits exceptional probability calibration, achieving an ECE of 0.029 and Brier score of 0.129, compared to 0.103 for GPT-5.6 and 0.599 for Qwen3-8B (detailed ECE curves in Appendix C). Over 95% of repeated states yield identical decisions under Jev (consistency 0.985).

Local Models and Energy Footprints: On identical A100-80GB hardware, the open reproduction Kev demonstrates a clear scaling trajectory: Kev-0.8B achieves 27.3% accuracy at 29.3 J net energy (47.2 J gross); Kev-4B reaches 32.1% at 100.0 J net (132.4 J gross); Kev-9B reaches 40.1% at 130.0 J net (170.5 J gross). Qwen3-8B achieves 39.6% accuracy but consumes 167.1 J net (211.2 J gross) and displays severe overconfidence (ECE 0.599). Kev-9B matches Qwen3-8B's accuracy while consuming 22% less net energy (energy frontier curve in Appendix C).

Two-Stage Cascades: To bridge the accuracy gap between Jev and GPT-5.6, we evaluate a confidence-based cascade: Jev evaluates the state first; if its confidence falls below threshold t, the state is escalated to GPT-5.6. Sweeping thresholds across 40 cross-validation folds on held-out tasks identifies an optimal threshold of $t = 0.65$. In live execution, the cascade achieves **84.9% strict accuracy** [82.8%, 86.9%] while escalating only **16.7% of decisions**. This recovers approximately half of the frontier model's accuracy lead while operating at Jev's median latency (162 ms) and reducing cost by 79% relative to GPT-5.6 alone ($\$8.16 \times 10^{-4}$ vs. $\$3.83 \times 10^{-3}$).

Table 1. Decision replay benchmark on 1,076 airline states (5× repeats, 5,380 decisions per decider). 95% CIs are cluster bootstrapped across states. Median and p95 latencies reflect client wall-clock time. GPU energy is measured on a dedicated NVIDIA A100-80GB GPU. †GPT-5.6 is evaluated against its own recorded actions, serving as an upper bound.

Decider	Strict Acc.	Lenient Acc.	p50 Lat.	p95 Lat.	$/Decision	ECE	Net Energy (J)
GPT-5.6†	88.5% [86.8, 90.1]	89.7% [88.1, 91.2]	3909 ms	11444 ms	3.83×10^{-3}	0.103	*hosted*
Jev → GPT-5.6 (t=0.65)	84.9% [82.8, 86.9]	86.0% [84.0, 88.0]	162 ms	3184 ms	8.16×10^{-4}	0.063	*hosted*
Jev	81.4% [79.1, 83.6]	82.6% [80.4, 84.8]	176 ms	251 ms	1.73×10^{-4}	0.029	*hosted*
GPT-OSS-20B	67.0% [64.6, 69.3]	68.6% [66.3, 70.9]	1299 ms	6069 ms	3.11×10^{-4}	0.286	*hosted*
Qwen3-8B (vLLM)	39.6% [36.8, 42.5]	44.2% [41.3, 47.2]	656 ms	1076 ms	*local*	0.599	167.1 ± 0.0
Kev-9B	40.1% [37.1, 43.1]	54.6% [51.5, 57.7]	548 ms	1053 ms	*local*	0.179	130.0 ± 0.0
Kev-4B	32.1% [29.4, 35.0]	44.5% [41.5, 47.6]	439 ms	832 ms	*local*	0.124	100.0 ± 0.0
Kev-0.8B	27.3% [24.7, 30.0]	41.8% [38.8, 44.7]	218 ms	384 ms	*local*	0.088	29.3 ± 0.0
Kev-4B → GPT-OSS	64.7% [62.3, 67.2]	66.3% [63.9, 68.7]	2879 ms	8747 ms	2.11×10^{-4}	0.311	*hybrid*

Table 2. In-the-loop agent performance on Tau2-Bench Airline (250 episodes per arm). 95% CIs are cluster bootstrapped across tasks.

Arm	Reward [95% CI]	Pass^5	$/Success	Sec/Turn
A (Jev → G)	0.744 [0.636, 0.844]	0.600	$0.0477	2.76 s
B (G → G)	0.796 [0.696, 0.888]	0.660	$0.1760	5.36 s
C (G alone)	0.792 [0.696, 0.884]	0.640	**$0.0333**	**2.63 s**
D (Small LLM)	0.692 [0.572, 0.804]	0.540	$0.0432	5.35 s
E (Kev-4B)	0.045 [0.012, 0.089]	0.000	$0.4759	3.24 s

3.3 In-the-Loop Agent Performance (Arms A–E)

While offline replay demonstrates that Jev is fast, cheap, and well-calibrated, real agent loops introduce dynamic multi-turn interactions where tool selections alter downstream environment state. We construct an in-loop experiment comparing five architectural configurations:

- **Arm A (Jev → G):** Jev selects the tool; generator G (GPT-5.6) populates arguments and handles user turns.
- **Arm B (G decides → G fills):** G selects the tool in a constrained call; G fills arguments in a second call (isolating decision cost).
- **Arm C (G unified):** G decides and generates arguments in a single standard call (the industry baseline).
- **Arm D (Small LLM → G):** GPT-OSS-20B selects the tool; G fills arguments.
- **Arm E (Kev-4B → G):** Kev-4B selects the tool; G fills arguments.

Every arm is evaluated over 50 airline tasks across 5 trials ($N = 250$ episodes per arm), utilizing GPT-5.6 as both generator G and simulated customer.

Table 2 and Figure 2 (with per-arm task reward broken down in Appendix Figure 6) reveal critical systems trade-offs:

1. Decoupled Decisions Suffer a Coordination Tax: Comparing Arm A (Jev) against Arm B (G deciding separately), Jev reduces agent cost by $\$0.105$ per episode [$\$0.085$, $\$0.126$] ($p < 0.05$) and cuts wall-clock time by 24.2 s per episode [19.3, 29.2]. However, Arm A incurs a statistically significant 5.2-point reward drop (0.744 vs. 0.796, 95% CI $[-10.8, -0.4]$).

2. Unified Execution (Arm C) Dominates: Crucially, when comparing Arm A against Arm C (the production standard where G selects tools and generates arguments in a single turn), Arm C achieves both higher reward (0.792 vs. 0.744) and **lower monetary cost per success** ($\$0.0333$ vs. $\$0.0477$). Because generating tool arguments requires a frontier model invocation regardless, splitting decision into a preceding step adds latency ($+4.1$ s per episode) and duplicate input token processing.

3. Open Decider Cascading Failures (Arm E): In live execution, Arm E collapses to an end-task reward of 0.045. Diagnostic analysis shows that Kev-4B frequently selects `get_user_details` before the user has supplied customer credentials. When forced to populate arguments for this premature tool call, generator G extracts fallback placeholder entities from tool docstrings (e.g., `sara_doe_496`), triggering consecutive environment errors that terminate 106 of 250 episodes under `too_many_errors`.

4 Decision Point 2: Semantic Cache Guarding

Semantic prompt caching seeks to eliminate redundant LLM inference by serving previously generated answers to semantically equivalent queries (Schroeder et al., 2026). In contrast to agent control, where generation is inevitable, cache guarding represents a scenario where the decision *replaces* generation: a correct reuse skips an expensive model call, while an incorrect reuse serves corrupted output.

4.1 Workloads and Benchmark Flaws

We evaluate semantic cache guarding across three diverse query streams ($N = 2,000$ test queries, $N = 600$ dev queries each):

1. **GSM-Plus (Li et al., 2024):** 2,000 algorithmic variations of grade-school math problems. Subtle perturbations (e.g., modifying numerical constants or reversing question direction) fundamentally alter the correct solution, while lexical phrasing remains near-identical. Reuse is correct for only 33.6% of test queries. Critically, **the median cosine similarity of wrong reuses (**0.966**) exceeds that of correct reuses (**0.955**)**, rendering standard vector similarity blind to errors by construction.

2. **vCache LM-Arena (Schroeder et al., 2026; Chiang et al., 2024):** 2,000 multi-turn chat prompts derived from Chatbot Arena, where variants were generated by an LLM. Reuse is valid for 92.5% of test queries.

3. **vCache SearchQueries (Schroeder et al., 2026; Craswell et al., 2020):** 2,000 real-world web search queries from the ORCAS dataset. Reuse is valid for 58.2% of test queries.

4.2 Diagnostic and Correction of Benchmark Labels

During initial evaluations, every decider—including the frontier GPT-5.6 model—consistently hit an apparent ceiling of 70–72% accuracy on SearchQueries and LM-Arena, in sharp contrast to 93–97% accuracy on GSM-Plus (where ground truth was established by exact mathematical verification).

Manual auditing of disagreement instances revealed that the benchmark labels from vCache (Schroeder et al., 2026) suffered from systemic construction flaws:

- **The Transitive Closure Fallacy:** In vCache's SearchQueries dataset, semantic classes were constructed via union-find over pairwise judgments emitted by a lightweight model (GPT-4.1-nano). If query $A \sim B$ and $B \sim C$, union-find transitively merges A and C into the same class, even if A and C are mutually contradictory (e.g., chaining "excel count blank cells" $\sim$ "excel count cells" $\sim$ "excel count non blank cells").

- **Unchecked Variant Generation:** In LM-Arena, variants generated from a seed query inherited the seed's class ID regardless of semantic drift (e.g., reversing logic premises or updating certification exam versions from SY0-501 to SY0-701). Conversely, 37 pairs with identical string prompts were assigned different class IDs.

To establish rigorous ground truth, we developed an independent, non-transitive pairwise relabeling protocol. Every pair was evaluated blind in two independent passes by Claude Opus 5.5 under a strict formal semantic guideline (`GUIDELINE.md`). A label was changed if and only if both passes independently agreed at high confidence. This corrected **6.0% of LM-Arena pairs** (156 of 2,600) and **15.7% of SearchQueries pairs** (409 of 2,600). Crucially, the cache guard query was formulated strictly as a query-equivalence decision ("Do these two queries represent the same request?"), rather than having the decider grade the stored answer's completeness.

4.3 Empirical Results

Table 3 and Figure 3 (with per-benchmark accuracy bar charts in Appendix Figure 7) report performance across the corrected benchmarks:

1. Deciders Conquer Adversarial Perturbations (GSM-Plus): On GSM-Plus, where vector embeddings fail to distinguish altered numerical constants, the tuned similarity threshold and vCache are forced to set an infinite threshold on dev, resulting in an effective accuracy of 66.1% (regenerating all queries, achieving 0% cache utility). In contrast, Jev achieves **93.3% accuracy** with only 1.1% wrong reuses. Paired bootstrap testing confirms a massive $+27.2$ point advantage for Jev over similarity thresholds [$+25.5$, $+29.1$] ($p < 0.05$) and $+11.7$ points over the trained floor classifier [$+10.1$, $+13.3$].

2. Embedding Heuristics Suffice for Standard Paraphrases (LM-Arena): On LM-Arena, where semantic equivalence corresponds to standard conversational paraphrasing, dense embeddings capture the semantic overlap effectively. A dev-tuned similarity threshold achieves 96.9% accuracy. Jev achieves 96.3% ($p =$ n.s.). However, a **Threshold $\rightarrow$ Jev Cascade** achieves **97.4% accuracy** [96.6%, 98.1%], statistically outperforming the threshold alone ($+0.5$ points, $p < 0.05$). The cascade delegates clear cases (sim < 0.678 or sim ≥ 0.844) to the zero-cost threshold and queries Jev on **only 3.0% of queries**, incurring an effective cost of just $\$5.91 \times 10^{-7}$ per decision.

3. Web Search Traffic (SearchQueries): On SearchQueries, Jev achieves 85.6% accuracy versus 81.2% for the similarity threshold ($+4.4$ points paired difference [$+2.3$, $+6.4$], $p < 0.05$). More importantly, Jev reduces the catastrophic wrong-reuse rate from 14.4% down to 3.8% (a paired reduction of -10.6 points, $p < 0.05$). Jev matches GPT-5.6 (85.6% vs. 85.2%, tie) at 1/48th the monetary cost ($\$1.83 \times 10^{-5}$ vs. $\$8.77 \times 10^{-4}$).

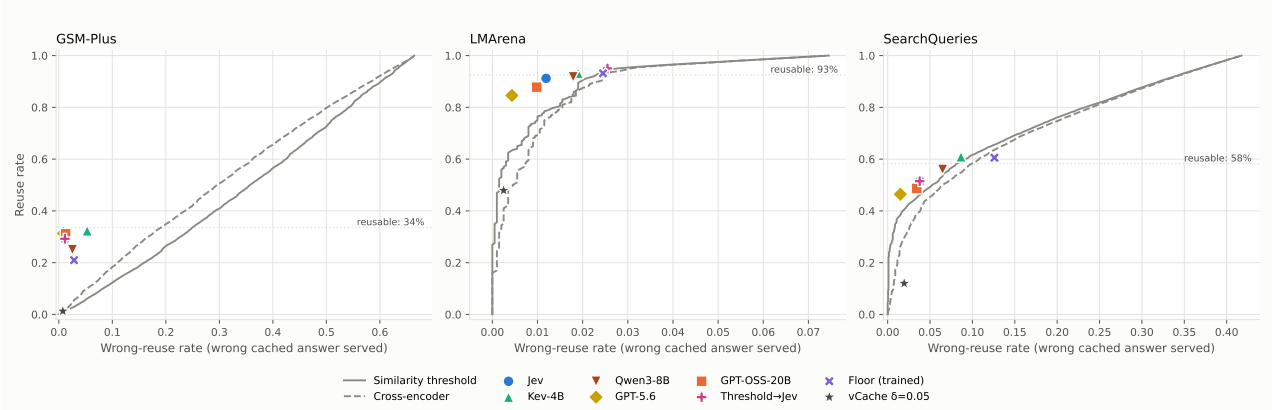


Figure 3. Cache-guard operating curves: reuse rate versus wrong-reuse rate (up and to the left is better). Solid lines show similarity thresholds and cross-encoders swept across all cutoffs; markers represent static decider operating points. On GSM-Plus, all deciders sit far above the similarity curve; on SearchQueries, deciders achieve high reuse with low error.

Table 3. Cache guard evaluation across 2,000 test queries per workload. 95% CIs are cluster bootstrapped across query classes. Wrong reuse denotes serving an incorrect cached answer; needless regen denotes regenerating an answer that could have been safely reused.

	GSM-Plus (Math Variations)			LM-Arena (Chat Paraphrases)			SearchQueries (Web Queries)		
Decider	**Acc.**	**Wrong R.**	**Need. Reg.**	**Acc.**	**Wrong R.**	**Need. Reg.**	**Acc.**	**Wrong R.**	**Need. Reg.**
Jev	93.3%	1.1%	5.5%	96.3%	1.2%	2.5%	85.6%	3.8%	10.6%
Thresh. → Jev	**93.3%**	1.1%	5.5%	**97.4%**	2.5%	0.1%	**85.7%**	3.8%	10.5%
GPT-5.6	96.0%	0.9%	3.2%	91.2%	0.4%	8.4%	85.2%	1.5%	13.3%
GPT-OSS-20B	95.0%	1.3%	3.7%	93.3%	1.0%	5.7%	83.6%	3.4%	13.0%
Kev-4B (local)	88.1%	5.3%	6.6%	96.9%	1.9%	1.3%	85.4%	8.6%	6.0%
Qwen3-8B (local)	86.3%	2.5%	11.1%	95.7%	1.8%	2.6%	84.8%	6.5%	8.7%
Similarity Thresh.	66.1%	0.3%	33.6%	96.9%	2.9%	0.3%	81.2%	14.4%	4.4%
Floor (Trained)	81.7%	2.9%	15.5%	95.7%	2.5%	1.8%	77.1%	12.6%	10.3%
vCache (δ=0.05)	66.1%	0.7%	33.1%	54.6%	0.3%	45.2%	50.3%	1.9%	47.7%

4.4 End-to-End Economic System Impact

To quantify the financial reality of semantic caching, we measure the true generation cost of answering un-cached queries using GPT-5.6 ($N = 200$ measured responses per dataset: GSM-Plus costs \$0.0016 / 4.3 s per answer; LM-Arena costs \$0.0084 / 13.8 s per answer). The net expected cost per query is:

$$\mathbb{E}[\text{Cost}] = C_{\text{decider}} + P(\text{regenerate}) \cdot C_{\text{gen}} \qquad (2)$$

Relative to an un-cached system (100% baseline cost):

- On GSM-Plus, similarity thresholds and vCache achieve 99–100% of baseline cost (saving nothing). Jev reduces system cost to **72%** (a net 28% savings).
- **Frontier LLMs are Self-Defeating Cache Guards:** Querying GPT-5.6 as a cache guard on GSM-Plus costs \$0.0011 per decision, resulting in an expected system cost of **136%**—deploying the guard costs 36% more than simply generating an answer every time!
- On LM-Arena, where 92.5% of queries can be reused, the similarity threshold cuts cost to 5%, the Threshold → Jev cascade achieves 6%, Jev achieves 32%, and GPT-5.6 costs 55%.

5 Decision Point 3: Model Routing

In heterogeneous model serving architectures, dynamic model routing directs incoming requests to the smallest, least expensive model capable of answering correctly, escalating to a frontier model only when necessary (Chen et al., 2023; Ong et al., 2024; Hu et al., 2024). Like cache guarding, routing represents an architectural point where the decision *replaces* generation by preempting an expensive model invocation.

5.1 Experimental Setup: RouterBench

We evaluate model routing on the RouterBench 0-shot benchmark (Hu et al., 2024). We focus on the canonical model pairing examined in the routing literature (Ong et al.,

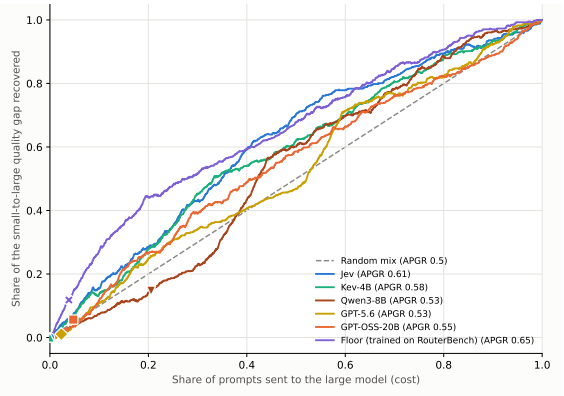


Figure 4. Model router trade-off curves: quality gap recovered versus large-model call fraction on RouterBench. Markers represent deciders' discrete default choices; curves sweep decision thresholds. The trained floor classifier outperforms all general deciders.

2024): a high-efficiency small model (**Mixtral-8x7B-chat**) versus a high-capacity large model (**GPT-4-1106-preview**).

The benchmark spans 1,998 test prompts and 600 dev prompts stratified across six diverse reasoning and knowledge benchmarks: MMLU, GSM8K, ARC-Challenge, HellaSwag, WinoGrande, and MBPP. An incoming prompt is labeled `large` if and only if Mixtral fails and GPT-4 succeeds; otherwise, it is labeled `small` (either because Mixtral succeeds, or because both models fail, rendering escalation futile). Exactly 28.3% of test prompts require the large model.

We measure performance using:

- **Quality:** The mean benchmark accuracy achieved by the routed model (Mixtral alone achieves 0.589 at $\$1.23 \times 10^{-4}$ per query; GPT-4 achieves 0.792 at $\$3.82 \times 10^{-3}$).
- **Area Preserved / Gap Recovered (APGR):** The integrated share of the small-to-large quality gap ($0.792 - 0.589 = 0.203$) recovered as the routing threshold is swept across all call fractions. A random mixture of models achieves an APGR of exactly 0.500.
- **Dev-Tuned Operating Point:** Following standard production practice, we tune a decision threshold on the dev split to target a $\approx 30\%$ escalation rate (matching the true positive rate of 28.3%), and apply this fixed threshold to test.

5.2 Empirical Results

Table 4 and Figure 4 summarize routing performance:

1. Discrete Choices Fail to Route: When queried for their discrete categorical choice ("small" vs. "large"), general-purpose deciders almost never escalate. Jev and Kev-4B escalate to GPT-4 on exactly **0.0% of prompts**. GPT-5.6 escalates on 2.3%, GPT-OSS-20B on 4.7%, and the trained floor on 3.9%. Consequently, utilizing discrete deciders as black-box categorical classifiers yields zero routing benefit; routing is viable only when operating on continuous confidence distributions $\hat{p}(\text{large})$ thresholded on a validation split.

2. Task-Agnostic Deciders Struggle: Among general-purpose deciders operating via threshold sweeping, Jev achieves the highest APGR (0.609 [0.585, 0.642]), recovering $+10.9$ points over a random mix ($p < 0.05$) and outperforming GPT-5.6 by $+8.0$ points ($p < 0.05$). Kev-4B achieves an APGR of 0.582. Strikingly, frontier LLMs perform near chance: GPT-5.6 achieves an APGR of only 0.529, barely exceeding the 0.500 random baseline.

3. The Primacy of Empirical Error Profiling: The domain-specific floor classifier—a lightweight `distilroberta` model trained on RouterBench's 20,000 prompt training split—achieves the highest APGR across all models: **0.649** [0.621, 0.678]. Paired bootstrap analysis confirms that the trained floor statistically outperforms Jev by **+0.040 points** [$+0.001$, $+0.070$] ($p < 0.05$). At the dev-tuned operating point ($\approx 28\%$ escalation), the floor classifier achieves a test quality of 0.691 ($+4.5$ points over a matched random mix) at a low cost of $\$7.55 \times 10^{-4}$ per query.

5.3 Why General Deciders Fall Short in Routing

The contrasting success of typed deciders in cache guarding versus their modest performance in model routing reveals a fundamental systems distinction:

- **Input-Intrinsic Semantics vs. Model-Relative Error Surfaces:** Cache guarding is an *input-intrinsic* task: whether two queries request the same underlying information is an objective semantic property fully visible in the input text. In contrast, model routing is an *error-surface prediction* task: determining whether Mixtral will fail on a specific question requires knowledge of Mixtral's idiosyncratic training distribution, tokenizer limitations, and domain weaknesses.

- **The Semantic Fallacy in Routing:** A prompt that appears elementary to a general semantic judge (e.g., Jev or GPT-5.6) may trigger subtle reasoning failures in Mixtral, while a seemingly dense technical question may fall squarely within its pre-training strengths. Because these failure surfaces are external to the prompt's semantic difficulty, general-purpose deciders cannot reliably predict them without direct empirical exposure to the target model's historical error logs.

Table 4. Model routing evaluation on RouterBench (1,998 test prompts, small = Mixtral-8x7B, large = GPT-4). APGR measures the integrated quality gap recovered (random mix = 0.500). Dev-tuned operating points fix the threshold on the dev split targeting a $\approx 30\%$ escalation rate. Paired differences against random and Jev are cluster bootstrapped across prompts.

Router	APGR [95% CI]	Diff vs. Random	Diff vs. Jev	Test Sent Large	Test Quality	$/Query
Always Small (Mixtral)	—	—	—	0.0%	0.589 [0.589, 0.589]	$\$1.23 \times 10^{-4}$
Always Large (GPT-4)	—	—	—	100.0%	0.792 [0.792, 0.792]	$\$3.82 \times 10^{-3}$
Floor (Trained on Train)	**0.649** [0.621, 0.678]	**+0.149** [+0.121, +0.178]	**+0.040** [+0.001, +0.070]	28.3%	**0.691** [0.673, 0.710]	$\$7.55 \times 10^{-4}$
Jev	0.609 [0.585, 0.642]	+0.109 [+0.085, +0.142]	—	28.4%	0.674 [0.655, 0.693]	$\$9.32 \times 10^{-4}$
Kev-4B (local)	0.582 [0.553, 0.609]	+0.082 [+0.053, +0.109]	-0.027 [−0.055, −0.007]	29.4%	0.678 [0.660, 0.697]	$\$1.16 \times 10^{-3}$
GPT-OSS-20B	0.546 [0.516, 0.576]	+0.046 [+0.016, +0.076]	-0.063 [−0.102, −0.034]	30.0%	0.668 [0.649, 0.687]	$\$1.32 \times 10^{-3}$
Qwen3-8B (local)	0.531 [0.488, 0.540]	+0.031 [−0.012, +0.040]	-0.078 [−0.130, −0.067]	22.7%	0.624 [0.605, 0.643]	$\$1.67 \times 10^{-3}$
GPT-5.6	0.529 [0.510, 0.568]	+0.029 [+0.010, +0.068]	-0.080 [−0.106, −0.045]	23.9%	0.649 [0.630, 0.669]	$\$2.13 \times 10^{-3}$

6 Cross-Cutting Analysis and Synthesis

6.1 When Does Deciding Without Generating Pay Off?

Comparing the empirical results across agent control, cache guarding, and model routing reveals a clear set of governing conditions (synthesized in Table 5):

1. The Preemption vs. Pipeline Principle: The fundamental determinant of decider utility is whether the decision *replaces* or merely *precedes* generation. Let C_{dec} denote decider invocation cost, C_{gen} denote generation cost, and $P(\text{skip})$ denote the probability that the decision successfully averts generation. The net expected cost delta of inserting a decider into a pipeline is:

$$\Delta\text{Cost} = C_{\text{dec}} - P(\text{skip}) \cdot C_{\text{gen}} \tag{3}$$

In agent control, $P(\text{skip}) = 0$ because generator G must still be invoked to synthesize tool parameters and customer messages. Consequently, $\Delta\text{Cost} = C_{\text{dec}} > 0$ strictly increases system cost and adds network latency. In contrast, in semantic cache guarding, $P(\text{skip}) = P(\text{reuse}) \in [0.3, 0.9]$. When $C_{\text{dec}} \ll C_{\text{gen}}$ (e.g., Jev at $\$2 \times 10^{-5}$ vs. GPT-5.6 generation at $\$2 \times 10^{-3}$, a $100\times$ ratio), deciders unlock massive net savings.

2. The Frontier LLM Inversion: Equation (7) explains why utilizing a frontier LLM as a system decider is catastrophic. If $C_{\text{dec}} \approx C_{\text{gen}}$, any non-zero rate of false reuses or needless regenerations renders $\Delta\text{Cost} > 0$. On GSM-Plus, querying GPT-5.6 as a cache guard inflated overall system cost to 136% of unconditional generation.

3. Semantic Nuance vs. Heuristic Blindness: When $P(\text{skip}) > 0$, deciders are justified only when the input contains semantic distinctions that cheaper heuristics cannot resolve. On LM-Arena, where prompt variation is primarily lexical paraphrasing, cosine similarity over dense embeddings achieves 96.9% accuracy at zero marginal cost. A dedicated decider is justified only in adversarial or entity-dense domains (GSM-Plus and SearchQueries) where lexical similarity diverges from semantic intent.

4. Input Intrinsicness vs. Error-Surface Profiling: When decisions depend on system-external variables—such as predicting which specific prompts will trigger reasoning failures in an external 8B model—semantic deciders perform near chance. Routing requires empirical calibration against the target model's historical error surface, favoring lightweight domain-trained classifiers over universal deciders.

6.2 Commercial Deciders vs. Open-Weight Reproductions

Our study directly compares proprietary RLCD deciders (Jev) against open-weights pointer reproductions (Kev-0.8B, 4B, 9B). In agent control, Kev falls substantially short of Jev (32.1% vs. 81.4% offline accuracy) and suffers from catastrophic looping in live loops (reward 0.045), showing that lightweight LoRA adapters struggle with multi-turn state machines without extensive preference tuning. Conversely, in semantic cache guarding, Kev-4B achieves remarkable parity with Jev (88.1% vs. 93.3% on GSM-Plus, 96.9% vs. 96.3% on LM-Arena, and 85.4% vs. 85.6% on SearchQueries), establishing that open-weight deciders are highly effective local alternatives for pairwise semantic matching.

6.3 Physical Hardware Energy vs. Monetary Pricing

Deploying local deciders on dedicated A100-80GB GPUs with NVML metering bridges commercial API pricing with physical resource consumption. At enterprise electricity rates ($\approx \$0.10/\text{kWh}$), Kev-4B's net energy consumption of 100.0 J (2.78×10^{-5} kWh) incurs an energy cost of just $\$2.78 \times 10^{-6}$ **per decision**. Factoring in server hardware amortization ($\approx \$1.50$–$\$2.50/\text{hour}$ for an A100 node), Kev-4B's 439 ms execution amortizes to $\approx \$6.5 \times 10^{-5}$ per call. Commercial API pricing for Jev ($\$1.8 \times 10^{-5}$ to $\$1.7 \times 10^{-4}$) closely matches the hardware amortization cost of self-hosting, while eliminating cluster management overhead.

Table 5. Cross-cutting synthesis across the three evaluated decision points. The deciding-without-generating paradigm succeeds when the decision replaces downstream generation and resolves semantic nuances invisible to lightweight heuristics.

Decision Point	Replaces Gen.?	Input Sufficiency	Leading Decider	System Payoff	Primary Failure Mode
Agent Control	**No** (G fills args)	Complete	Unified G alone (Arm C)	**Negative** (+$0.014/succ.)	Extra call; duplicate prompt tokens
Cache: GSM-Plus	**Yes** (skips gen.)	Complete	Jev / Cascade	**Large** (-28% total cost)	Similarity blind to math alterations
Cache: LM-Arena	**Yes** (skips gen.)	Redundant	Threshold $\rightarrow$ Jev	**Moderate** (cascade 97.4%)	Over-engineering simple paraphrases
Cache: Search	**Yes** (skips gen.)	Complete	Jev / Cascade	**Substantial** (cuts error 11%)	High false-reuse rate under threshold
Model Router	**Yes** (skips gen.)	**Incomplete**	Trained Floor	**Marginal** ($+4.5\%$ quality)	Model failure surfaces external to text

7 Related Work

Compound AI Systems and Agent Control: The shift from monolithic models to compound AI systems (Zaharia et al., 2024) has motivated modular agent architectures where models interact with environments via tool calling (Schick et al., 2023; Mialon et al., 2023; Yao et al., 2024). Hierarchical systems like TinyAgent (Chen et al., 2024) and AgentFlux (Wang et al., 2025) explore function routing to minimize execution cost. Our work provides the first controlled systems comparison of specialized, non-autoregressive deciders versus unified generative agents in multi-turn loops.

Semantic Prompt Caching: Semantic caching reduces LLM serving costs by reusing responses across semantically equivalent queries (Schroeder et al., 2026). Existing systems typically rely on embedding cosine thresholds (Reimers & Gurevych, 2019; Zhang et al., 2024) or learned per-prompt error bounds δ (Schroeder et al., 2026). However, embedding similarity fails on fine-grained reasoning and math word problems (Li et al., 2024). We demonstrate that typed deciders solve these failure modes, and our audit resolves transitive label propagation flaws in standard caching benchmarks.

LLM Routing and Model Cascades: Dynamic routing between heterogeneous models optimizes cost-performance trade-offs (Chen et al., 2023; Ong et al., 2024; Hu et al., 2024). Classic architectures explore early exiting through cascades (Teerapittayanon et al., 2016; Wang et al., 2018). Our evaluation shows that general semantic deciders cannot replace empirical error profiling when predicting target-model failures.

Non-Autoregressive and Typed Models: Constrained generation frameworks enforce schema constraints over autoregressive outputs but retain full self-attention and KV-cache overhead. Standalone decision architectures—such as Type-Safe Jev and open reproduction Kev—eliminate autoregression entirely by directly outputting probability vectors over discrete options. This work provides the first empirical evaluation of their operational trade-offs.

8 Conclusion

This paper has presented the first independent systems evaluation of the *deciding-without-generating* paradigm across three foundational LLM-system decision points: agent control, semantic cache guarding, and model routing. Evaluating commercial non-autoregressive deciders (Jev), open reproductions (Kev), generative LLMs, and non-neural baselines on a shared harness, we establish a unifying systems principle: *Typed, non-autoregressive decision models deliver transformative efficiency gains when their decision replaces downstream generation and resolves semantic nuances invisible to cheap heuristics. They fail to pay off when inserted before generation that occurs anyway, or when predicting external model failure surfaces.*

For systems architects, our findings translate into four guidelines: (1) **Do Not Decouple Tool Selection in Agent Loops:** Unified frontier LLMs executing decision and argument synthesis in a single turn dominate decoupled pipelines in cost and task completion. (2) **Deploy Typed Deciders as Cache Guards:** In semantic caching, typed deciders eliminate false-reuse risks on complex queries (+27.2 points accuracy and 28% cost savings on GSM-Plus), avoiding self-defeating frontier LLM guards. (3) **Exploit Cascades for Hybrid Traffic:** Combining zero-cost embedding similarity with an ambiguous-band escalation to a typed decider establishes the empirical Pareto frontier, querying the decider on as few as 3% of requests. (4) **Profile Empirical Error Surfaces for Routing:** In model routing, generic semantic understanding cannot substitute for target-model error logs.

References

Chen, L., Zaharia, M., and Zou, J. FrugalGPT: How to use large language models while reducing cost and improving performance. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 36, pp. 73847–73873, 2023.

Chen, L., Zou, J., and Zaharia, M. TinyAgent: Function calling at the edge. *arXiv preprint arXiv:2409.00608*, 2024.

Chiang, W.-L., Zheng, Z., Lin, B., Liaw, R., Liang, P.,

Stoica, I., and Zheng, X. Chatbot arena: An open platform for evaluating LLMs by human preference. *International Conference on Machine Learning (ICML)*, 2024.

Craswell, N., Campos, D., Mitra, B., Yilmaz, E., and Billerbeck, B. ORCAS: 18 million clicked pairs for open search. In *ACM SIGIR Conference on Research and Development in Information Retrieval*, pp. 1289–1298, 2020.

Hu, Q., Bie, F., Roy, D., Stretcu, A., Liaw, R., and Stoica, I. RouterBench: A benchmark for multi-LLM routing. *arXiv preprint arXiv:2403.12031*, 2024.

Li, Q., Yin, B., Lan, H., Shen, Z., Cheng, Y., and Huang, F. GSM-Plus: A comprehensive benchmark for evaluating the robustness of LLMs on mathematical reasoning. *arXiv preprint arXiv:2402.19255*, 2024.

Mialon, G., Dessì, R., Lomeli, M., Nalmpantis, C., Pasunuru, R., Scialom, F., Rozière, B., Celikyilmaz, A., Touvron, H., Lacroix, T., et al. Augmented language models: a survey. *Transactions on Machine Learning Research*, 2023.

Ong, I., Chen, A., Shen, J., Desai, A., Zaharia, S., and Stoica, I. RouteLLM: Learning to route LLMs with preference data. *arXiv preprint arXiv:2406.18665*, 2024.

Reimers, N. and Gurevych, I. Sentence-bert: Sentence embeddings using siamese BERT-networks. In *Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 3982–3992, 2019.

Schick, T., Dwivedi-Yu, J., Dessì, R., Raileanu, R., Lomeli, M., Zettlemoyer, L., Cancedda, N., and Scialom, T. Toolformer: Language models can teach themselves to use tools. *Advances in Neural Information Processing Systems (NeurIPS)*, 36:68539–68551, 2023.

Schroeder, L. G., Desai, A., Cuadron, A., Chu, K., Liu, S., Zhao, M., Krusche, S., Kemper, A., Zaharia, M., and Gonzalez, J. E. vCache: Verified semantic prompt caching. In *International Conference on Learning Representations (ICLR)*, 2026.

Teerapittayanon, S., McDanel, B., and Kung, H. T. Branchynet: Fast inference via early exiting from deep neural networks. In *International Conference on Pattern Recognition (ICPR)*, pp. 2464–2469, 2016.

Wang, S., Zhou, K., Zhao, H., and Wen, J. AgentFlux: Hierarchical decision routing for efficient multi-agent systems. *arXiv preprint arXiv:2510.00229*, 2025.

Wang, X., Yu, F., Dou, Z.-Y., Darrell, T., and Gonzalez, J. E. Skipnet: Learning dynamic routing in convolutional networks. In *European Conference on Computer Vision (ECCV)*, pp. 409–424, 2018.

Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., and Cao, Y. Tau-bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*, 2024.

Zaharia, M., Khattab, O., Chen, L., Zou, J., Gonzalez, J. E., and Stoica, I. The shift from models to compound AI systems. *Communications of the ACM*, 2024.

Zhang, X., Zhang, Y., Long, X., Xie, F., and Dai, Z. Towards general text embeddings with multi-stage contrastive learning. *arXiv preprint arXiv:2405.06012*, 2024.

A Statistical Methodology and Bootstrapping

A.1 Cluster Bootstrap Protocol

In modern LLM evaluations, treating repeated model invocations on the same state or query as independent identically distributed (i.i.d.) observations severely underestimates standard errors and produces artificially narrow confidence intervals. Because model outputs on identical prompts exhibit high intra-cluster correlation, all confidence intervals in this work are computed via **cluster bootstrapping** with $B = 1,000$ resamples:

1. **Cluster Identification:** The clustering unit is defined by the underlying independent data generation process:
   - In *Agent Control (Arm 0)*, clusters are the 1,076 unique dialogue states.
   - In *In-the-Loop Agent Control*, clusters are the 50 distinct τ-bench tasks.
   - In *Cache Guarding*, clusters are the query equivalence classes: in SearchQueries, the `id_set`; in GSM-Plus, the seed question variant groups; in LM-Arena, the original prompt clusters.
   - In *Model Routing*, clusters are the 1,998 individual benchmark prompts.
2. **Resampling Procedure:** For each bootstrap iteration $b \in \{1, \ldots, B\}$, we sample clusters with replacement until the original number of clusters is reached. All trials or repeated calls belonging to a selected cluster are included in resample b.
3. **Paired Differences:** When evaluating decider A against decider B, we compute the paired difference $\Delta^{(b)} = \theta_A^{(b)} - \theta_B^{(b)}$ on the exact same resampled clusters. We report the empirical 2.5th and 97.5th percentiles of $\Delta^{(b)}$ as the 95% confidence interval. A difference is denoted statistically significant ($p < 0.05$) if its 95% CI strictly excludes zero.

B Prompt Formats and Decider Interfaces

B.1 Agent Control State Rendering

To ensure identical evaluation across all deciders, conversation histories in τ-bench airline are serialized into a standardized transcript format:

```
System policy: You are an airline
customer service agent. You must
follow airline policies...
Available tools:
[get_flight_details, get_reservation,
cancel_reservation, ...]
User: I need to cancel my flight
from Chicago to New York.
Assistant: Could you please provide
your reservation confirmation code?
User: Sure, it is ABC123.
```

For Jev and Kev, available tools are passed as candidate criteria:

```
{"get_reservation": "Retrieve
details of an existing booking",
"cancel_reservation": "Cancel
an active flight reservation",
"respond_to_user": "Send a natural
language response to the customer"}
```

For generative models (GPT-5.6, GPT-OSS-20B, and Qwen3-8B), the model is invoked via native tool-choice APIs constrained to the set of available tools, or via a single-turn structured prompt requiring a single tool identifier.

B.2 Cache Guard Decision Prompt

For the cache guard, deciders receive the incoming user query and the cached query (without the stored answer, following the query-only formulation in Section 4.2):

```
System: You are an expert semantic
cache guard. Determine whether
the new user query represents the
exact same underlying request as
the cached query, such that serving
the cached answer would fully and
accurately satisfy the user.
New Query: [Query A]
Cached Query: [Query B]
Options: [reuse, regenerate]
```

B.3 Model Router Decision Prompt

For RouterBench, deciders are provided with the prompt text and model specifications:

```
System: You are an expert LLM
routing system. Decide whether the
following prompt should be routed
to a small model (Mixtral-8x7B)
or escalated to a large frontier
model (GPT-4). Choose 'small' if
Mixtral can correctly solve it;
choose 'large' only if Mixtral will
fail and GPT-4 is required.
Prompt: [User Task Text]
Options: [small, large]
```

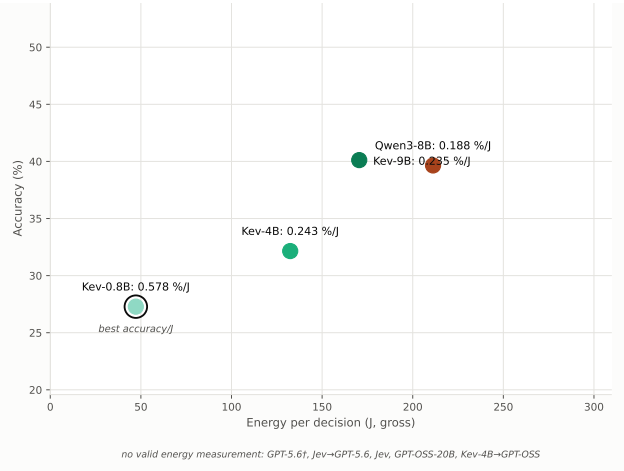


Figure 5. Accuracy versus net GPU energy per decision (Joules on NVIDIA A100-80GB) for local deciders in airline agent control. Kev-9B achieves comparable accuracy to Qwen3-8B while consuming 22% less net energy.

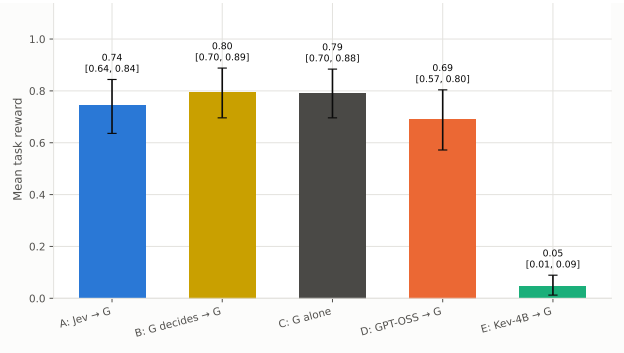


Figure 6. Task reward per arm in live agent loops (50 tasks $\times$ 5 trials, 250 episodes per arm). Arms B and C achieve $\approx$ 0.80 reward; Arm A (Jev) achieves 0.744.

C Extended Benchmark Results

C.1 Detailed Paired Comparisons for Cache Guarding

Table 6 provides the complete paired bootstrap difference tests across all three cache guarding benchmarks on identical test queries.

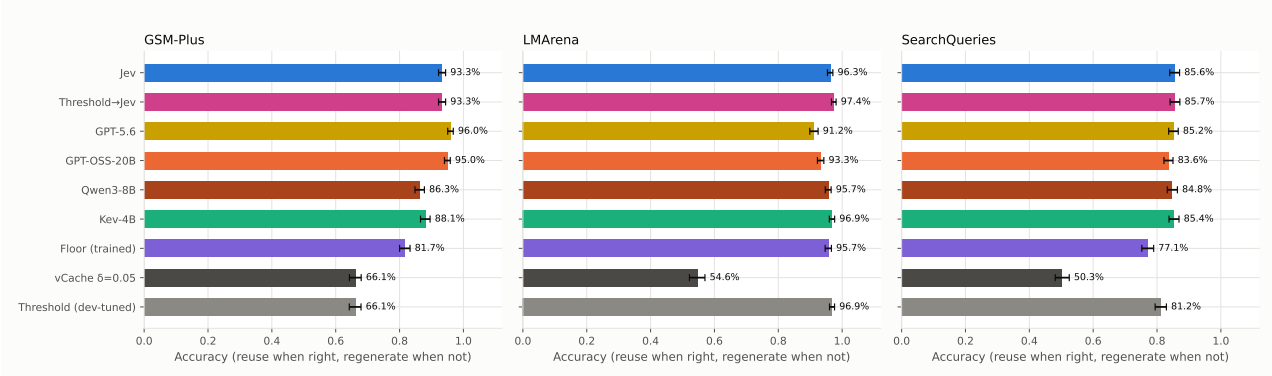


Figure 7. Cache-guard accuracy across GSM-Plus, LM-Arena, and SearchQueries (2,000 test queries each). Whiskers denote 95% CIs clustered by query class. On GSM-Plus, Jev outperforms embedding thresholds by $+27.2$ points; on LM-Arena, the Threshold $\rightarrow$ Jev cascade achieves 97.4% accuracy.

Table 6. Paired differences in accuracy and wrong-reuse share (percentage points, $A - B$) across 2,000 test queries per workload. Starred values (*) indicate 95% CIs excluding zero.

Comparison $(A - B)$	**Acc. Diff [95% CI]**	**Wrong Reuse Diff [95% CI]**
GSM-Plus (Math Variations)		
Jev − Threshold	$+27.2^*$ [+25.5, +29.1]	$+0.8^*$ [+0.3, +1.4]
Jev − vCache	$+27.2^*$ [+25.4, +29.0]	+0.4 [−0.2, +1.0]
Jev − Floor	$+11.7^*$ [+10.1, +13.3]	-1.7^* [−2.5, −0.9]
Jev − Kev-4B	$+5.2^*$ [+4.0, +6.6]	-4.2^* [−5.2, −3.2]
GPT-5.6 − Jev	$+2.6^*$ [+1.9, +3.4]	−0.3 [−0.6, +0.0]
LM-Arena (Chat Paraphrases)		
Threshold − Jev	+0.6 [−0.4, +1.6]	$+1.7^*$ [+1.2, +2.3]
Cascade − Threshold	$+0.5^*$ [+0.2, +0.9]	-0.4^* [−0.6, −0.1]
Floor − Threshold	-1.1^* [−1.9, −0.4]	-0.4^* [−0.9, −0.1]
Kev-4B − Jev	+0.6 [−0.3, +1.4]	$+0.7^*$ [+0.3, +1.1]
SearchQueries (Web Queries)		
Jev − Threshold	$+4.4^*$ [+2.3, +6.4]	-10.6^* [−12.1, −9.1]
Jev − Floor	$+8.5^*$ [+6.5, +10.5]	-8.8^* [−10.2, −7.3]
GPT-5.6 − Jev	−0.4 [−1.5, +0.8]	-2.3^* [−3.1, −1.6]

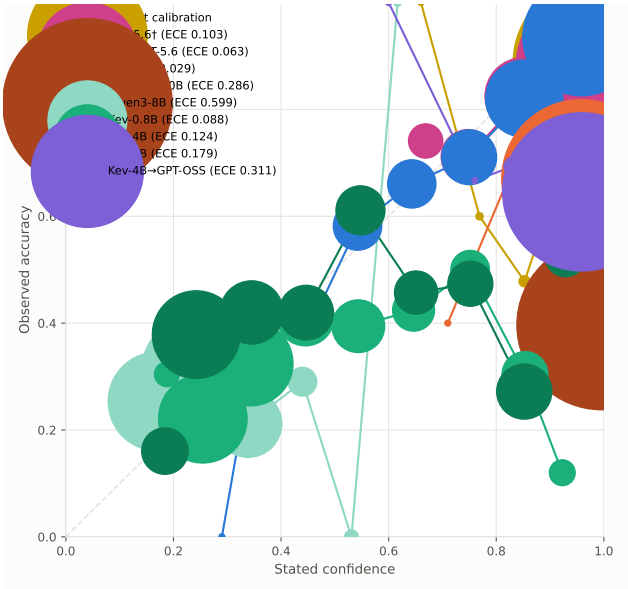


Figure 8. Expected Calibration Error (ECE) curves for airline agent control. Jev achieves near-perfect calibration (ECE $= 0.029$), whereas generative LLMs (Qwen3-8B and GPT-OSS-20B) exhibit substantial overconfidence.

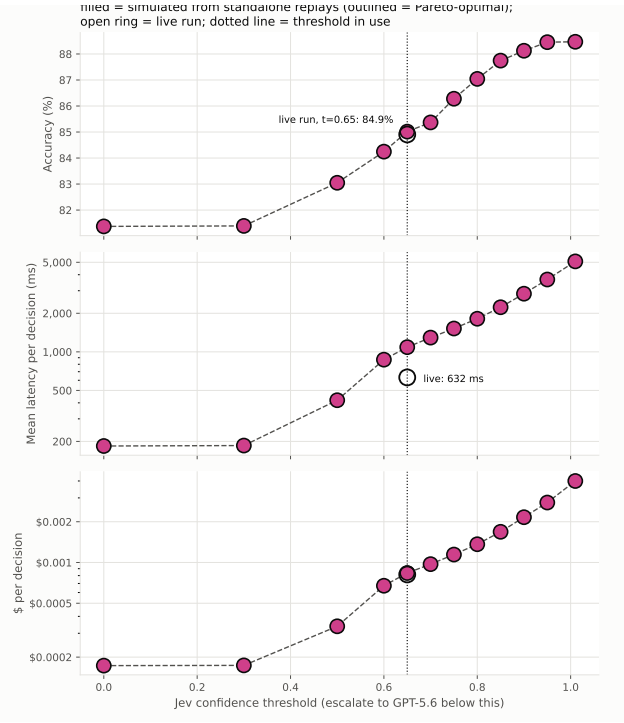


Figure 9. Offline sweep of the Jev → GPT-5.6 cascade threshold on held-out tasks. The selected threshold ($t = 0.65$) marks the Pareto-optimal operating point nearest the utopia point.